

Студент групи ІІІ-33 Соколов Олександр

Білет 52

Б52

1. Формалізуйте зв'язок 1:1. Застосуйте всі можливі варіанти умовності (1:1; 1:1y; 1y:1y) – як можна фізично реалізувати такі зв'язки? Чи можливо? Поясніть. Навести DDL скрипти та відпрацювати у власній СУБД.
2. «Інтернет-магазин». Навести перелік клієнтів, які ніколи не замовляли струни для гітари. Які сутності задіяні? Які між ними зв'язки? По якому полю будете поєднувати?

Практична робота

За предметною областю та завданням на запит, необхідно:

- розробити даталогічну модель нормалізовану, з певними обмеженнями цілісності та формалізованими зв'язками;

коротенький опис частини предметної області, що задіяна у запиті;

- написати запит за завданням по спроектованій даталогічній моделі;

- використати всі можливі варіанти

- INNER / OUTER JOIN;

- IN;

- EXISTS;

- операції над множинами INTERSECT or UNION or MINUS/EXCEPT.

Завдання 1

Зв'язок 1:1 – це такий тип зв'язку між двома сутностями, при якому кожному екземпляру першої сутності відповідає не більше одного екземпляра другої, і навпаки.

1:1 – Максимум одна пара з кожного боку, і участь обох сутностей обов'язкова.

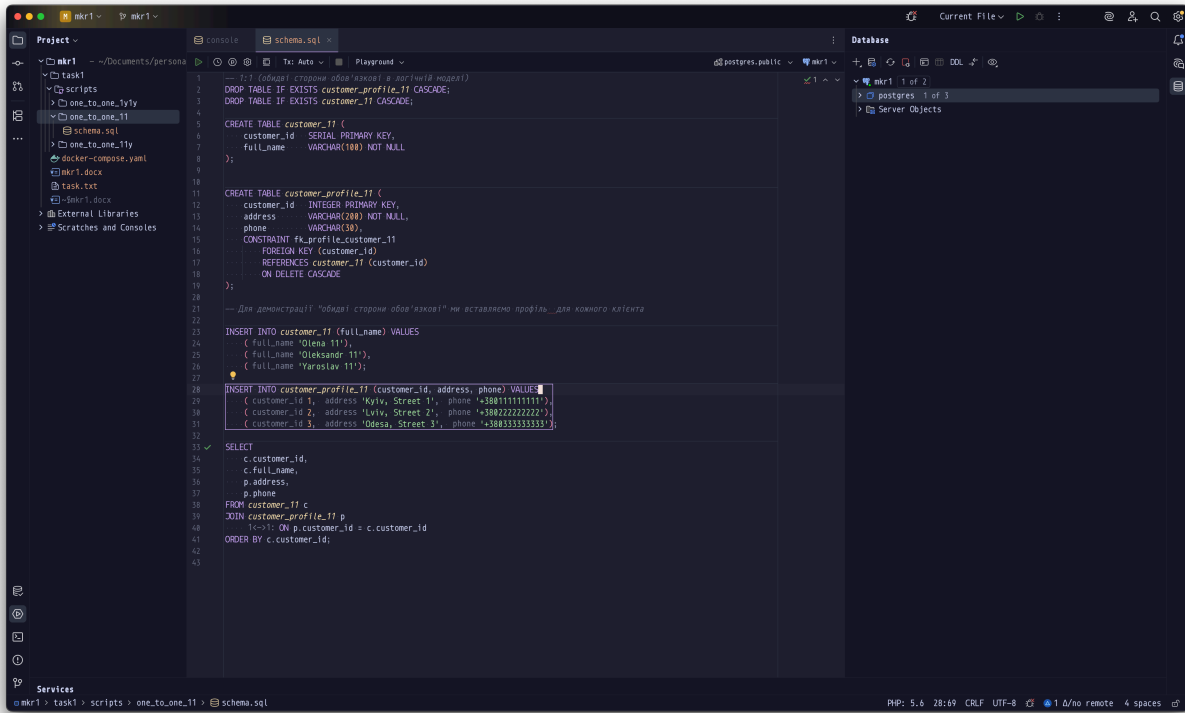
Фізично: у «дочірній» таблиці стовпець одночасно PRIMARY KEY і FOREIGN KEY на «батьківську», і бізнес-правила/NOT NULL не дозволяють існувати записам без пари.

1:1_y – Максимум одна пара з кожного боку, але для однієї сутності пара обов'язкова, для іншої – може бути, а може й ні.

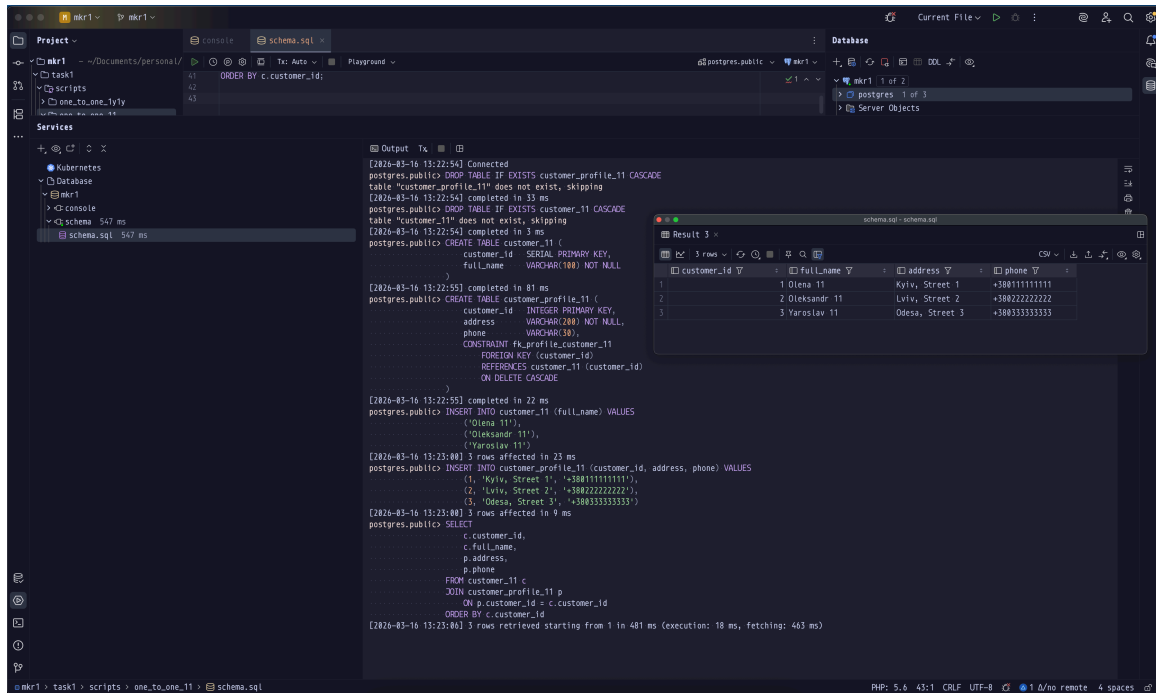
Фізично: той самий 1:1 через PK + UNIQUE/FK, але з боку «обов'язкової» сутності зовнішній ключ не може бути порожнім (NOT NULL + логіка), з іншого боку допускається відсутність пов'язаного запису.

1_y:1_y – Максимум одна пара з кожного боку, але для обох сутностей пара необов'язкова (0 або 1 зв'язаний екземпляр).

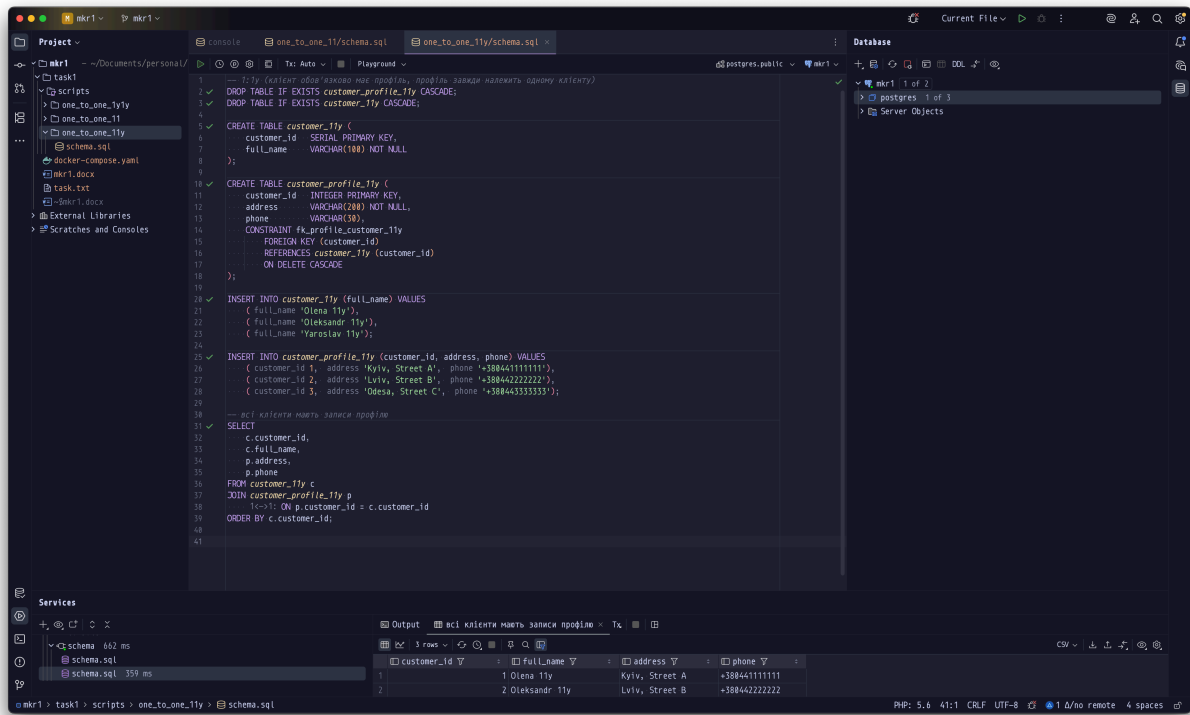
Фізично: 1:1 через PK + UNIQUE/FK, але допускається, що частина екземплярів однієї або другої сутності не має відповідного запису; це видно, наприклад, через LEFT JOIN, де поля другої сутності можуть бути NULL.



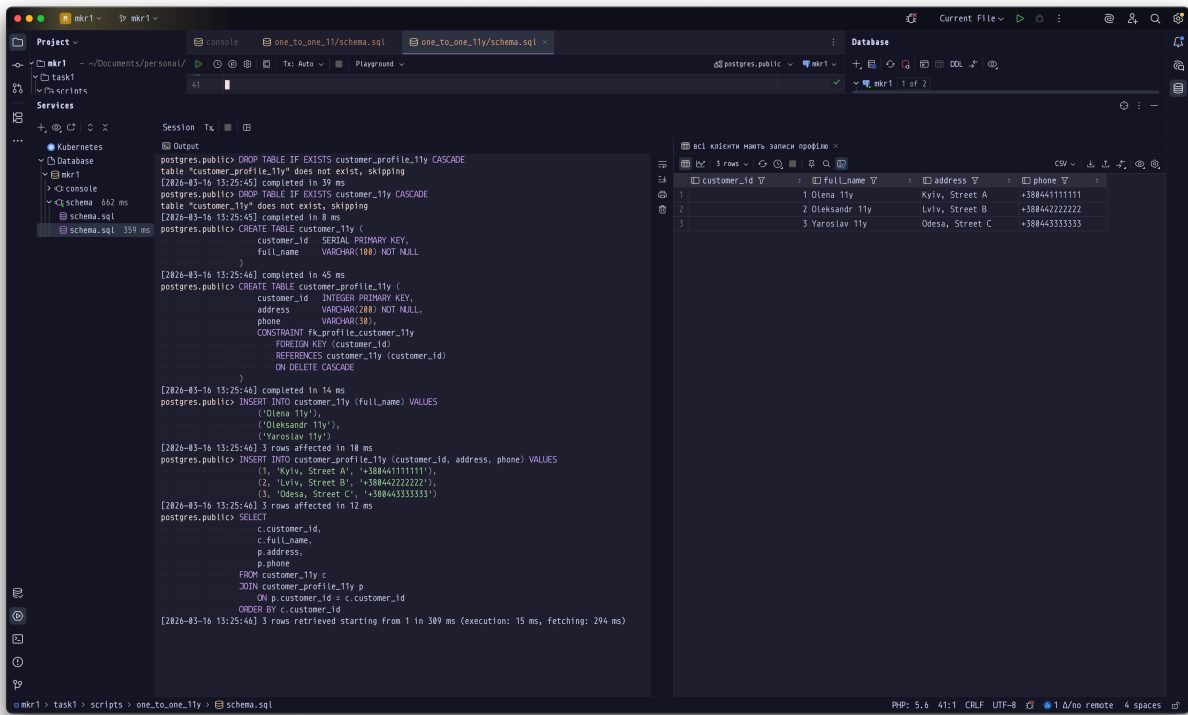
DDL для 1:1



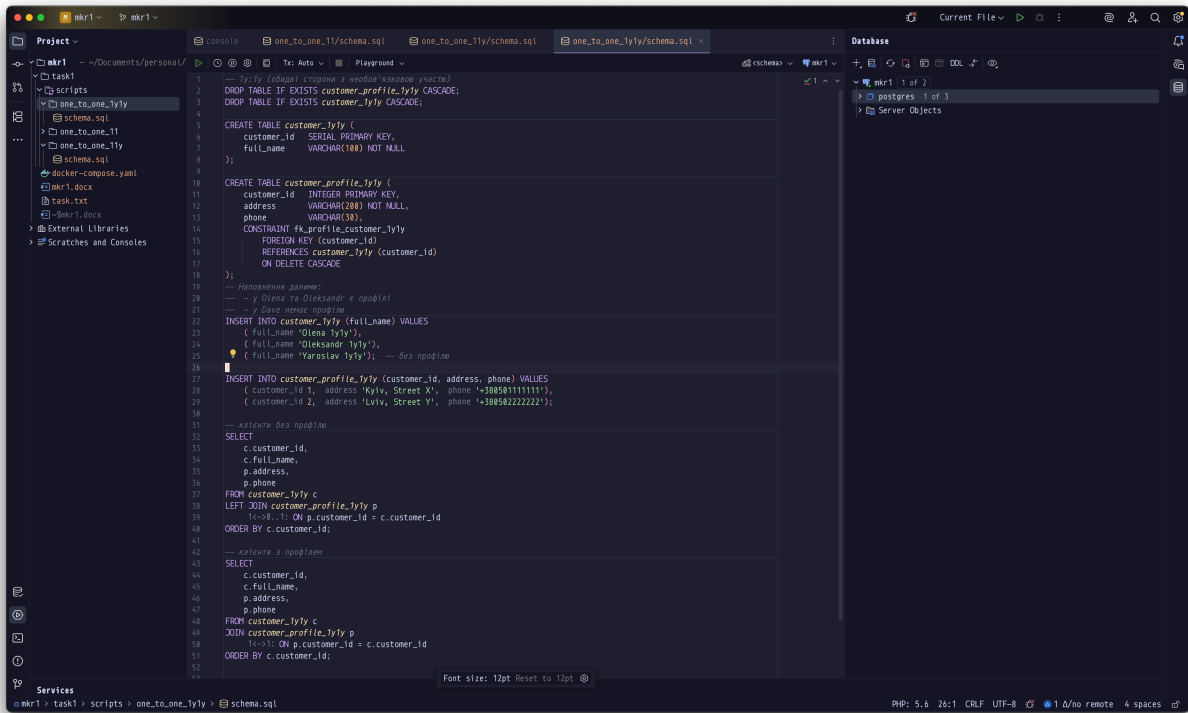
Відпрацювання скриптів та селекту для 1:1



DDL для 1:1y



Відпрацювання скриптів та селекту для 1:1y



DDL для 1y:1y

The screenshot shows a database IDE with a project named 'mkr1'. The 'Output' tab displays the execution of SQL scripts. The scripts create two tables: 'customer_tyly' and 'customer_profile_tyly'. The 'customer_tyly' table has columns 'customer_id' (SERIAL PRIMARY KEY), 'full_name' (VARCHAR(100) NOT NULL), 'address' (VARCHAR(200) NOT NULL), and 'phone' (VARCHAR(30)). The 'customer_profile_tyly' table has columns 'customer_id' (INTEGER PRIMARY KEY), 'address' (VARCHAR(200) NOT NULL), and 'phone' (VARCHAR(30)). It also has a foreign key constraint 'fk_profile_customer_tyly' referencing 'customer_tyly' and a 'ON DELETE CASCADE' clause.

The execution results show the following steps:

- Drop table 'customer_profile_tyly' if it exists (skipped).
- Create table 'customer_tyly' (completed in 36 ms).
- Create table 'customer_profile_tyly' (completed in 10 ms).
- Insert data into 'customer_tyly' (3 rows affected in 11 ms).
- Insert data into 'customer_profile_tyly' (2 rows affected in 13 ms).
- Select data from 'customer_tyly' and 'customer_profile_tyly' (3 rows retrieved starting from 1 in 417 ms).
- Select data from 'customer_tyly' and 'customer_profile_tyly' (2 rows retrieved starting from 1 in 289 ms).

Two windows show the data in the tables:

Клієнти без профілю:

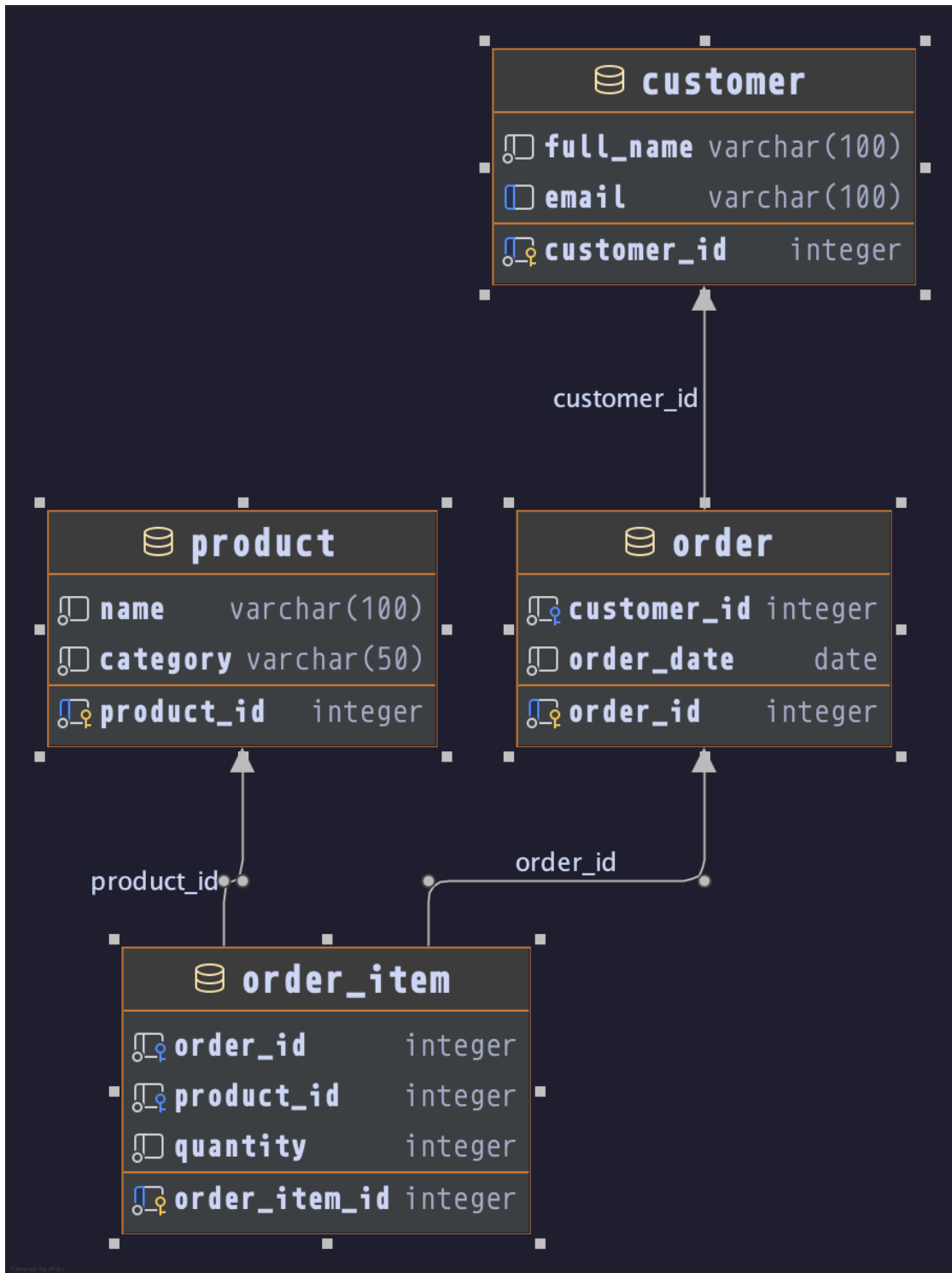
customer_id	full_name	address	phone
1	Olena Tyly	Kyiv, Street X	+380501111111
2	Oleksandr Tyly	Lviv, Street Y	+380502222222
3	Yaroslav Tyly		

Клієнти з профілем:

customer_id	full_name	address	phone
1	Olena Tyly	Kyiv, Street X	+380501111111
2	Oleksandr Tyly	Lviv, Street Y	+380502222222

Відпрацювання скриптів та селекту для 1y:1y

Завдання 2



Даталогічна модель

Зв'язки

CUSTOMER 1 : N ORDER

Один клієнт може мати багато замовлень, замовлення належить одному клієнту.

ORDER 1 : N ORDER_ITEM

Одне замовлення містить багато позицій, позиція належить одному замовленню.

PRODUCT 1 : N ORDER_ITEM

Один товар може входити в багато позицій замовлень, кожна позиція пов'язана з одним товаром.

```
mkr1 > task2 > scripts > schema.sql
```

```
1  DROP TABLE IF EXISTS order_item CASCADE;
2  DROP TABLE IF EXISTS "order" CASCADE;
3  DROP TABLE IF EXISTS product CASCADE;
4  DROP TABLE IF EXISTS customer CASCADE;
5
6  CREATE TABLE customer (
7      customer_id SERIAL PRIMARY KEY,
8      full_name VARCHAR(100) NOT NULL,
9      email VARCHAR(100) UNIQUE
10 );
11
12 CREATE TABLE product (
13     product_id SERIAL PRIMARY KEY,
14     name VARCHAR(100) NOT NULL,
15     category VARCHAR(50) NOT NULL
16 );
17
18 CREATE TABLE "order" (
19     order_id SERIAL PRIMARY KEY,
20     customer_id INTEGER NOT NULL,
21     order_date DATE NOT NULL,
22     CONSTRAINT fk_order_customer
23     FOREIGN KEY (customer_id)
24     REFERENCES customer (customer_id)
25 );
26
27 CREATE TABLE order_item (
28     order_item_id SERIAL PRIMARY KEY,
29     order_id INTEGER NOT NULL,
30     product_id INTEGER NOT NULL,
31     quantity INTEGER NOT NULL CHECK (quantity > 0),
32     CONSTRAINT fk_item_order
33     FOREIGN KEY (order_id)
34     REFERENCES "order" (order_id),
35     CONSTRAINT fk_item_product
36     FOREIGN KEY (product_id)
37     REFERENCES product (product_id)
38 );
39
```

DDL створення таблиць

```

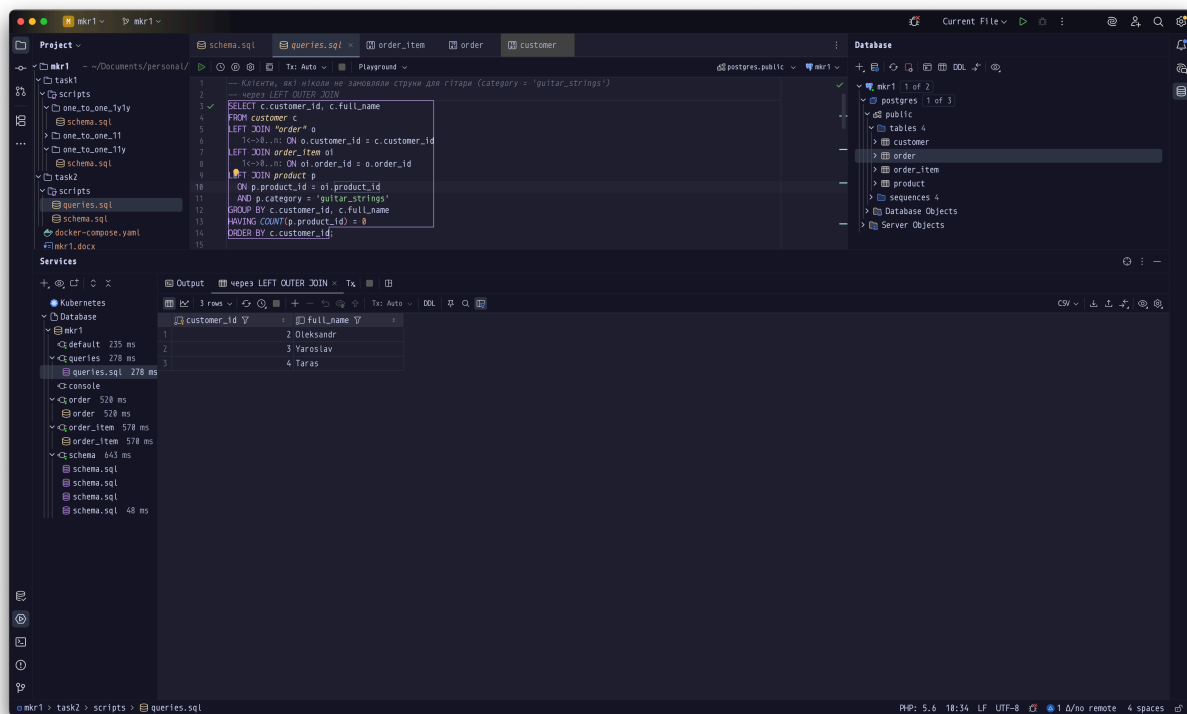
40  -- клієнти
41  INSERT INTO customer (full_name, email) VALUES
42  |... ('Olena', ..... 'olena@example.com'),
43  |... ('Oleksandr', .. 'oleksandr@example.com'),
44  |... ('Yaroslav', ... 'yaroslav@example.com'),
45  |... ('Taras', ..... 'taras@example.com');
46
47  -- товари
48  INSERT INTO product (name, category) VALUES
49  |... ('Electric guitar strings set', 'guitar_strings'),
50  |... ('Acoustic guitar strings set', 'guitar_strings'),
51  |... ('Guitar picks pack', ..... 'accessories'),
52  |... ('Drum sticks', ..... 'drums');
53
54  -- замовлення
55  INSERT INTO "order" (customer_id, order_date) VALUES
56  |... (1, DATE '2024-01-10'), ...-- Olena
57  |... (1, DATE '2024-02-05'), ...-- Olena
58  |... (2, DATE '2024-01-20'), ...-- Oleksandr
59  |... (3, DATE '2024-03-01'); ...-- Yaroslav
60
61  -- Taras (4) поки що без замовлень
62
63  -- позиції замовлень
64  -- Olena: замовляла і струни, і аксесуари
65  INSERT INTO order_item (order_id, product_id, quantity) VALUES
66  |... (1, 1, 1), ...-- Струни для електричної гітари
67  |... (1, 3, 2), ...-- Підвіски для електричної гітари
68  |... (2, 2, 1); ...-- Струни для акустичної гітари
69
70  -- Oleksandr: замовляв лише аксесуари (без струн)
71  INSERT INTO order_item (order_id, product_id, quantity) VALUES
72  |... (3, 3, 1); ...-- Набір медіаторів (аксесуари)
73
74  -- Yaroslav: замовляв щось інше, теж без струн
75  INSERT INTO order_item (order_id, product_id, quantity) VALUES
76  |... (4, 4, 2); ...-- Барабанні палички
77
78  -- Тепер:
79  -- ...-- Olena замовляла guitar_strings
80  -- ...-- Oleksandr і Yaroslav замовляли тільки не-guitar_strings
81  -- ...-- Taras взагалі нічого не замовляв

```

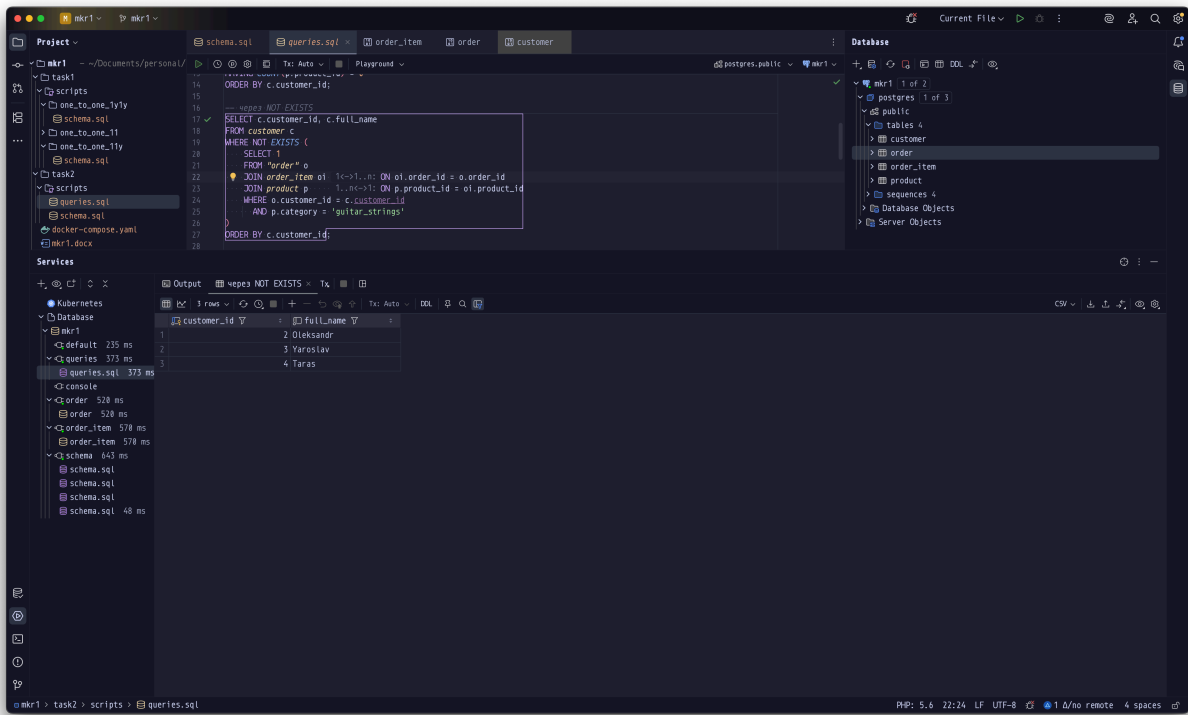
Наповнення бази даних

Опис предметної області

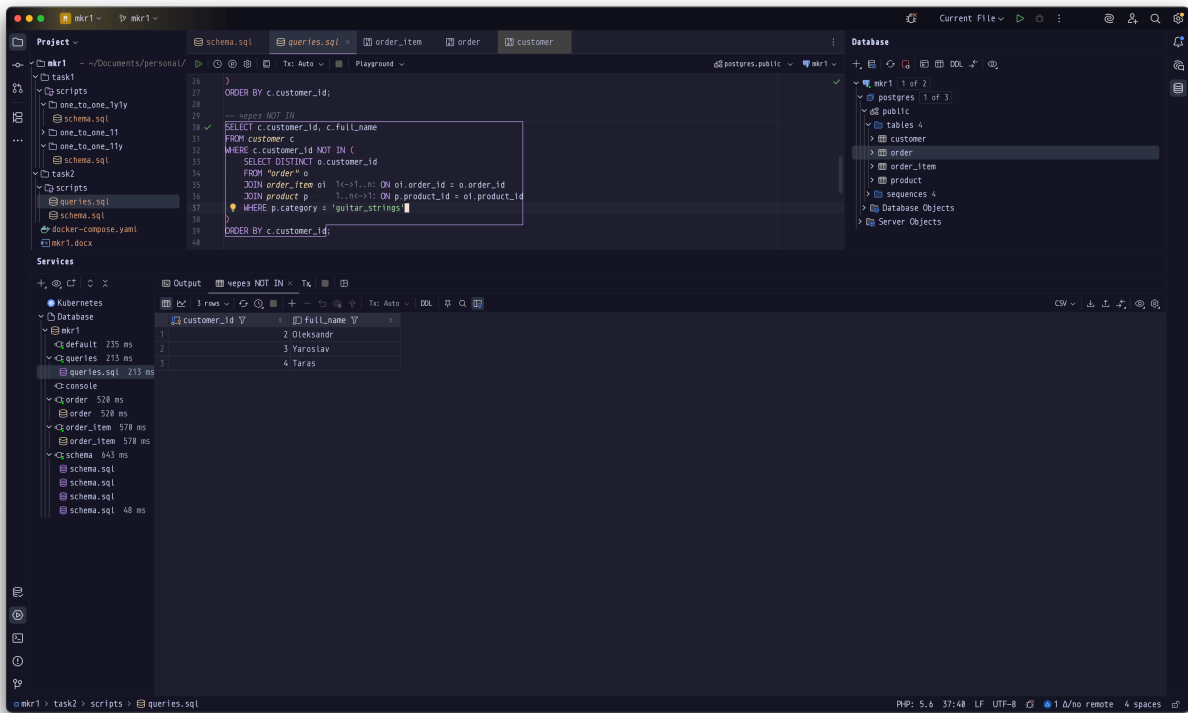
У розглянутій предметній області моделюється частина роботи інтернет-магазину музичних товарів. Клієнти роблять замовлення, кожне замовлення складається з однієї або декількох позицій, а кожна позиція прив'язана до конкретного товару. Товари поділяються на категорії, зокрема виділяється окрема категорія guitar_strings для струн до гітари. Запит працює саме з цими сутностями (клієнти, замовлення, позиції замовлень, товари) і дозволяє визначити клієнтів, які ніколи не замовляли товари з категорії струн для гітари.



Клієнти, які ніколи не замовляли струни для гітари (category = 'guitar_strings')
через LEFT OUTER JOIN



через NOT EXISTS



Через NOT IN

The screenshot shows a database IDE with a project named 'mkr1'. The main editor displays a SQL query in 'queries.sql' that uses the 'NOT IN' operator to find customers who have not placed orders. The query is as follows:

```
ORDER BY c.customer_id;
-- через EXCEPT (операция над множествами)
SELECT c.customer_id, c.full_name
FROM customer c
EXCEPT
-- Которые, но еще не заказывали guitar_strings
SELECT DISTINCT customer_id c2.customer_id, full_name c2.full_name
FROM customer c2
JOIN "order" o ON o.customer_id = c2.customer_id
JOIN "order_item" oi ON oi.order_id = o.order_id
JOIN product p ON p.product_id = oi.product_id
WHERE p.category = 'guitar_strings'
ORDER BY customer_id;
```

The 'Database' panel on the right shows the schema of the 'mkr1' database, including tables like 'customer', 'order', 'order_item', and 'product'. The 'Services' panel at the bottom shows the execution results of the query, displaying 3 rows of customer data.

customer_id	full_name
1	Oleksandr
2	Yaroslav
3	Taras

Через EXCEPT